

Amazon Route 53

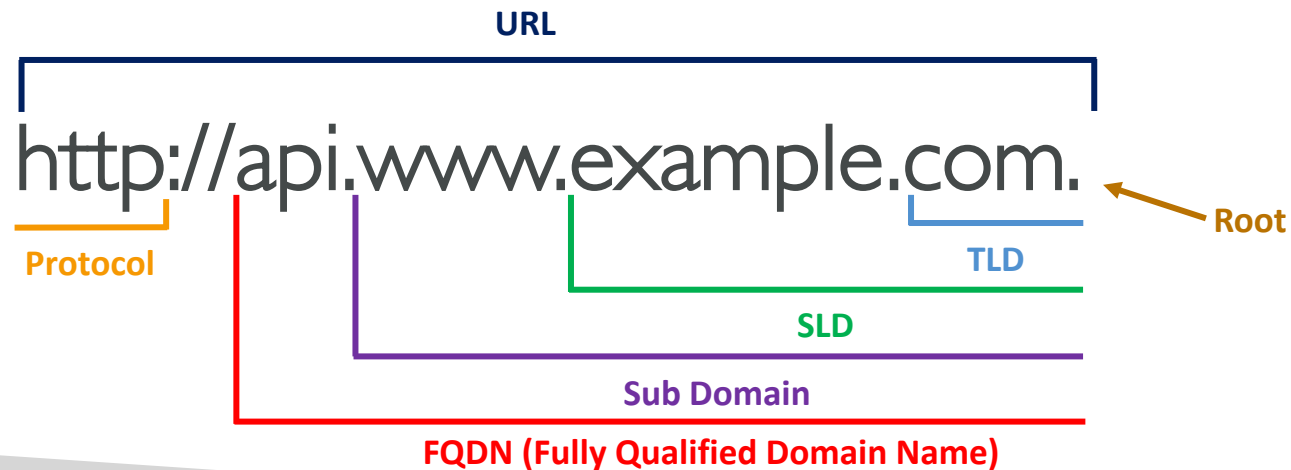
What is DNS?

- Domain Name System which translates the human friendly hostnames into the machine IP addresses
- `www.google.com => 172.217.18.36`
- DNS is the backbone of the Internet
- DNS uses hierarchical naming structure

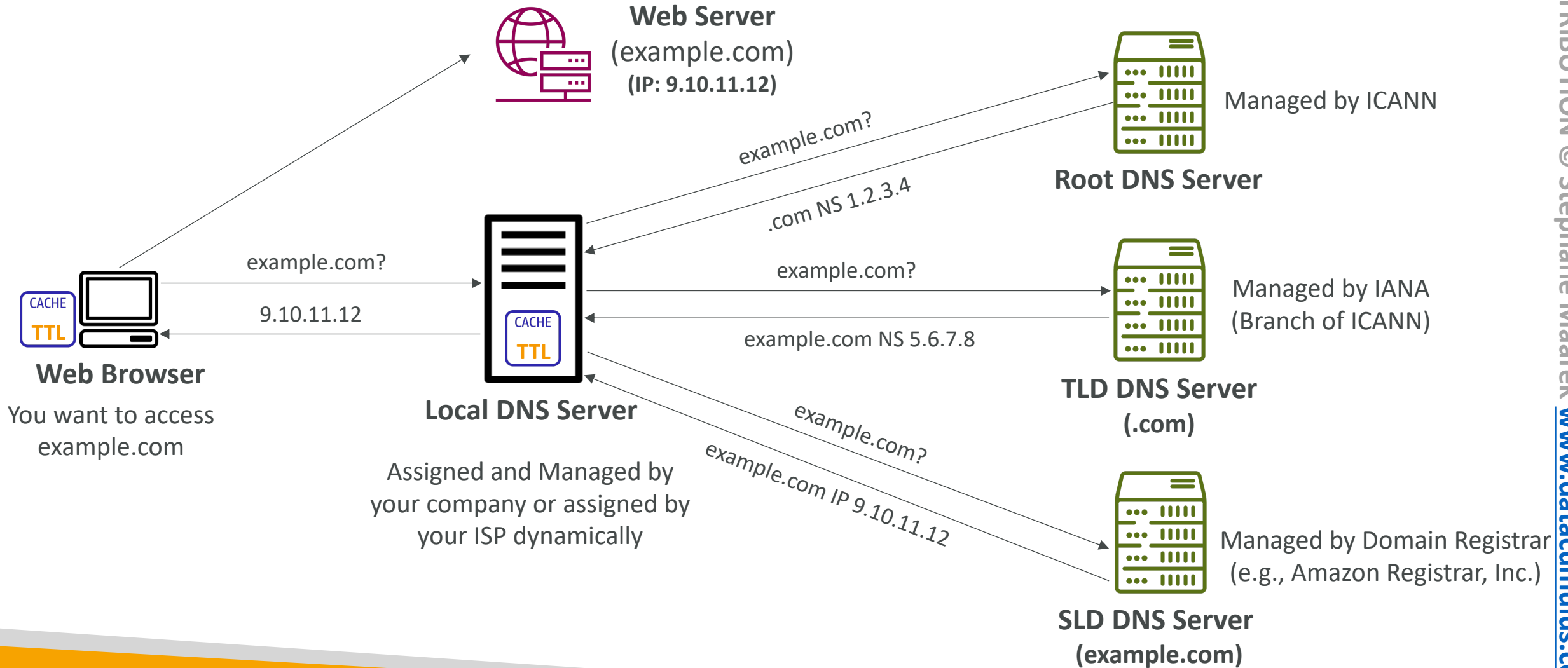
`.com`
`example.com`
`www.example.com`
`api.example.com`

DNS Terminologies

- Domain Registrar: Amazon Route 53, GoDaddy, ...
- DNS Records: A, AAAA, CNAME, NS, ...
- Zone File: contains DNS records
- Name Server: resolves DNS queries (Authoritative or Non-Authoritative)
- Top Level Domain (TLD): .com, .us, .in, .gov, .org, ...
- Second Level Domain (SLD): amazon.com, google.com, ...

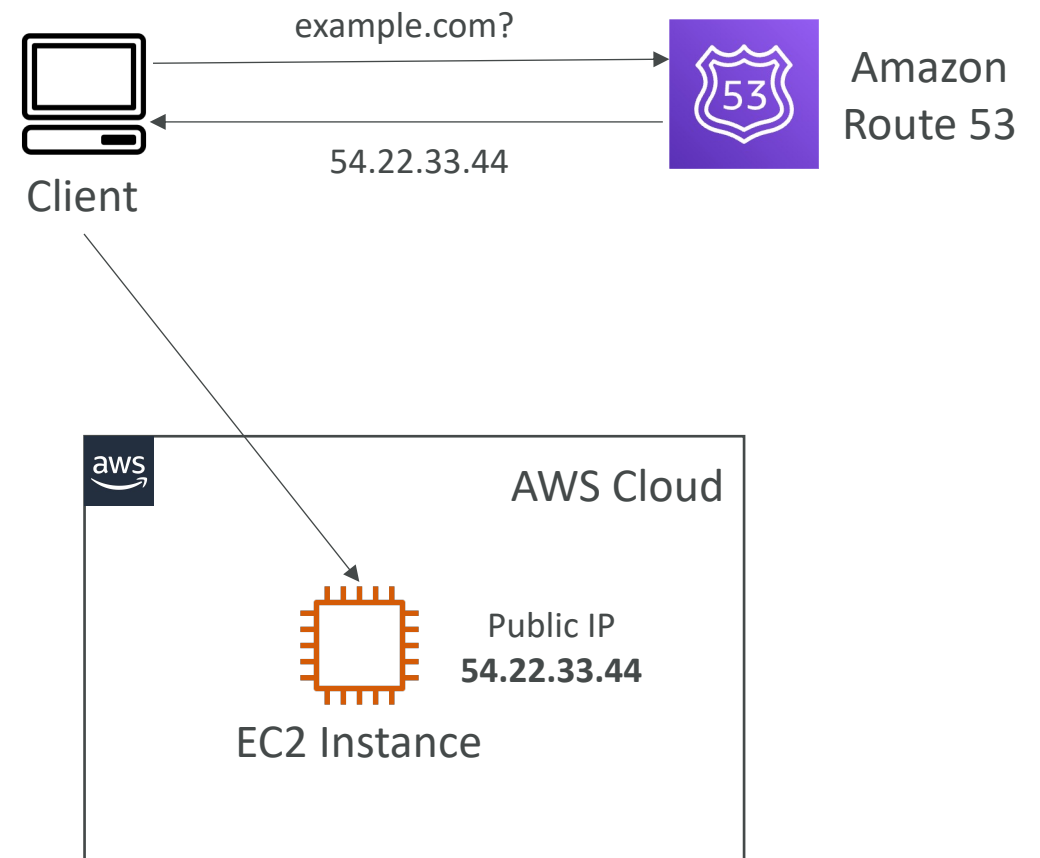


How DNS Works



Amazon Route 53

- A highly available, scalable, fully managed and *Authoritative* DNS
 - Authoritative = the customer (you) can update the DNS records
- Route 53 is also a Domain Registrar
- Ability to check the health of your resources
- The only AWS service which provides 100% availability SLA
- Why Route 53? 53 is a reference to the traditional DNS port

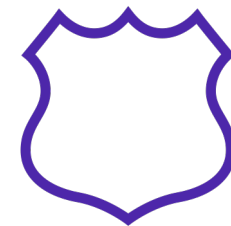


Route 53 – Records

- How you want to route traffic for a domain
- Each record contains:
 - **Domain/subdomain Name** – e.g., example.com
 - **Record Type** – e.g., A or AAAA
 - **Value** – e.g., 12.34.56.78
 - **Routing Policy** – how Route 53 responds to queries
 - **TTL** – amount of time the record cached at DNS Resolvers
- Route 53 supports the following DNS record types:
 - (must know) A / AAAA / CNAME / NS
 - (advanced) CAA / DS / MX / NAPTR / PTR / SOA / TXT / SPF / SRV

Route 53 – Record Types

- **A** – maps a hostname to IPv4
- **AAAA** – maps a hostname to IPv6
- **CNAME** – maps a hostname to another hostname
 - The target is a domain name which must have an A or AAAA record
 - Can't create a CNAME record for the top node of a DNS namespace (Zone Apex)
 - Example: you can't create for example.com, but you can create for www.example.com
- **NS** – Name Servers for the Hosted Zone
 - Control how traffic is routed for a domain

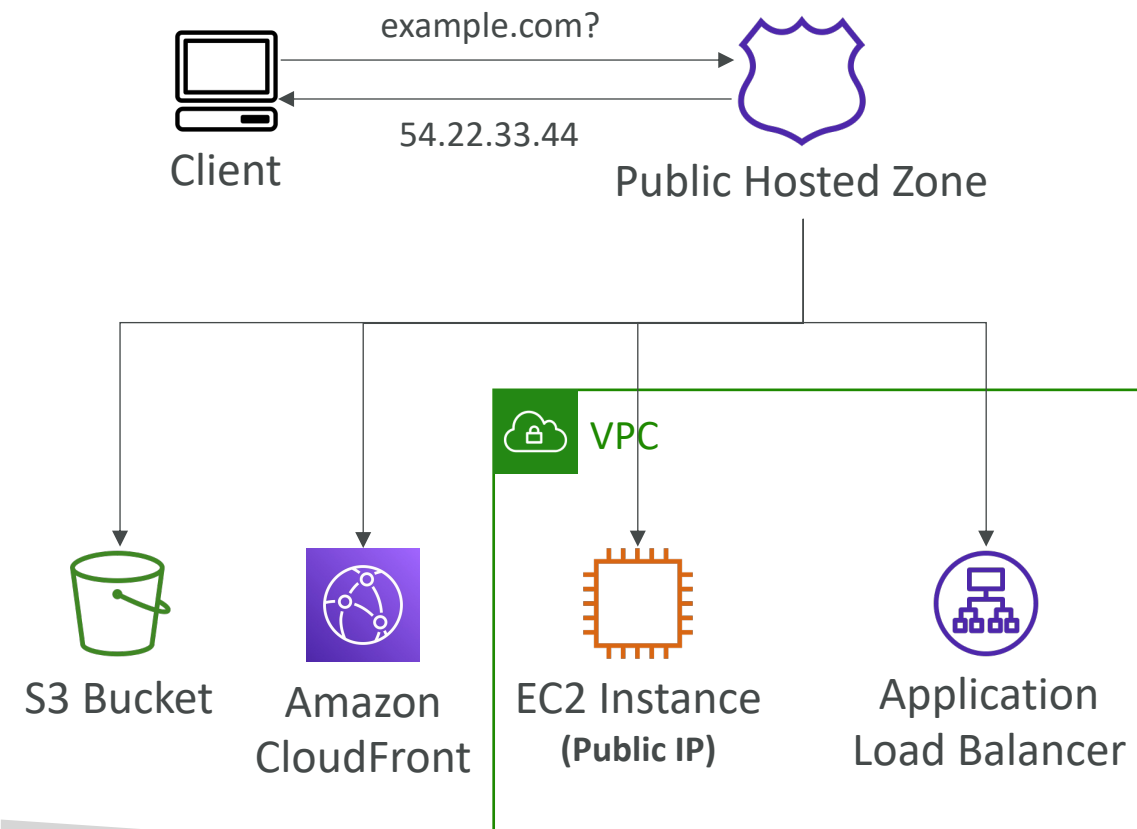


Route 53 – Hosted Zones

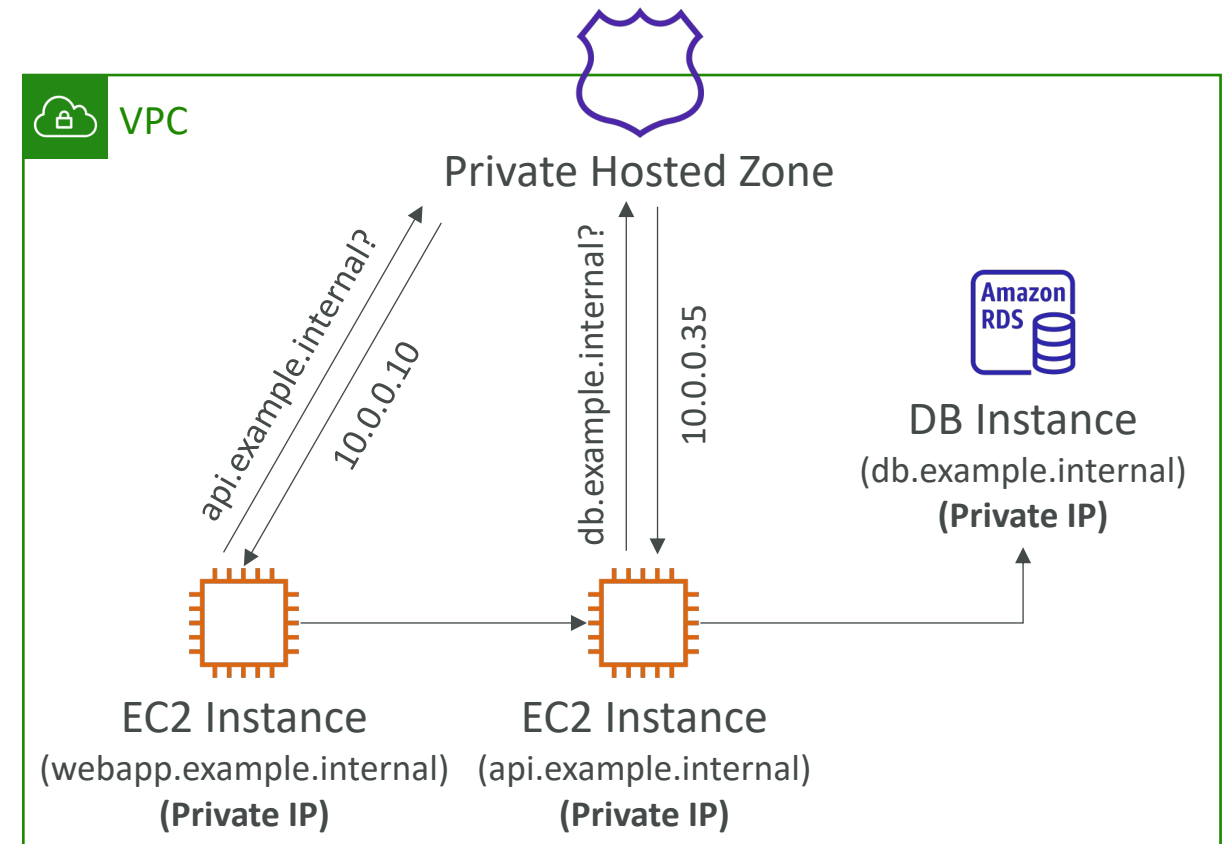
- A container for records that define how to route traffic to a domain and its subdomains
- **Public Hosted Zones** – contains records that specify how to route traffic on the Internet (public domain names)
[application1.mypublicdomain.com](#)
- **Private Hosted Zones** – contain records that specify how you route traffic within one or more VPCs (private domain names)
[application1.company.internal](#)
- You pay \$0.50 per month per hosted zone

Route 53 – Public vs. Private Hosted Zones

Public Hosted Zone

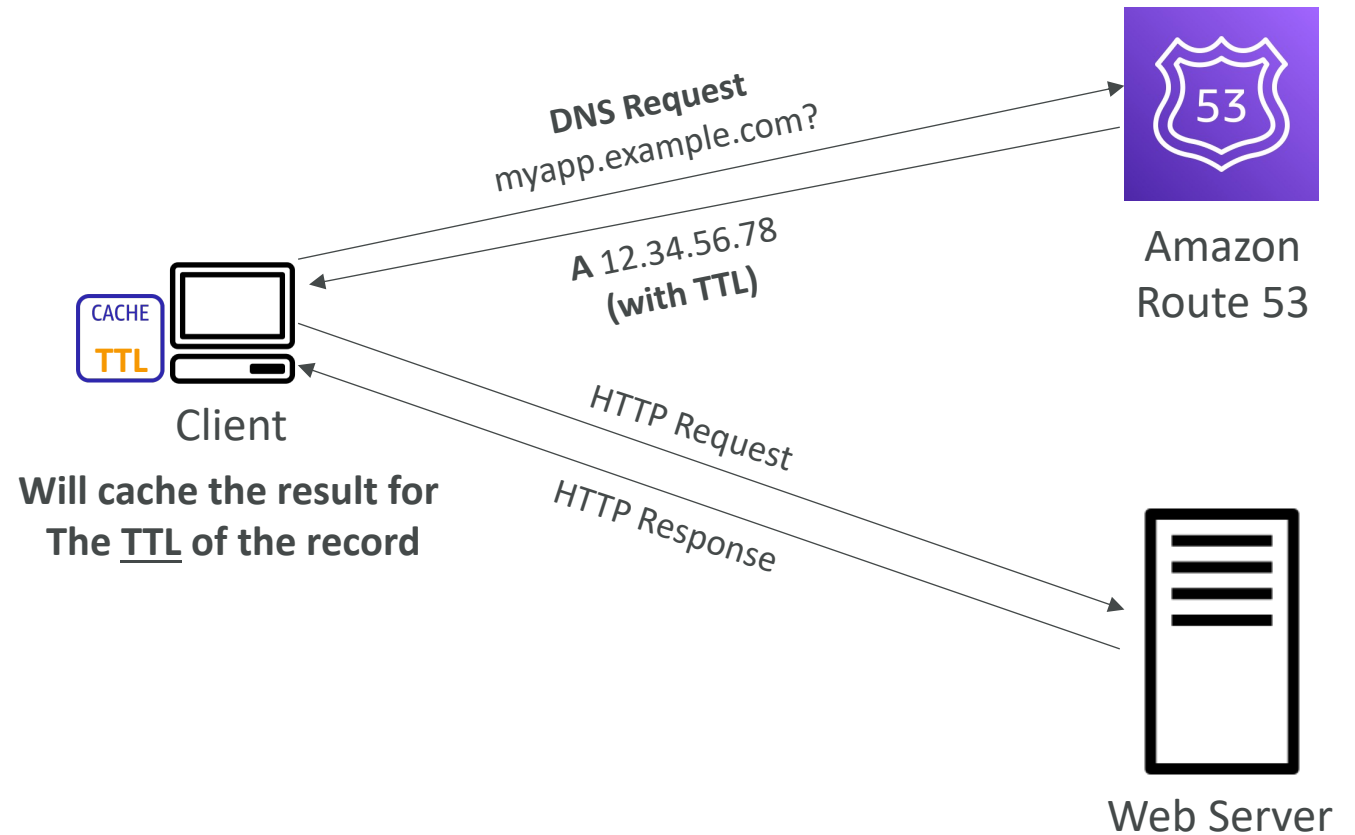


Private Hosted Zone



Route 53 – Records TTL (Time To Live)

- High TTL – e.g., 24 hr
 - Less traffic on Route 53
 - Possibly outdated records
- Low TTL – e.g., 60 sec.
 - More traffic on Route 53 (\$\$)
 - Records are outdated for less time
 - Easy to change records
- Except for Alias records, TTL is mandatory for each DNS record

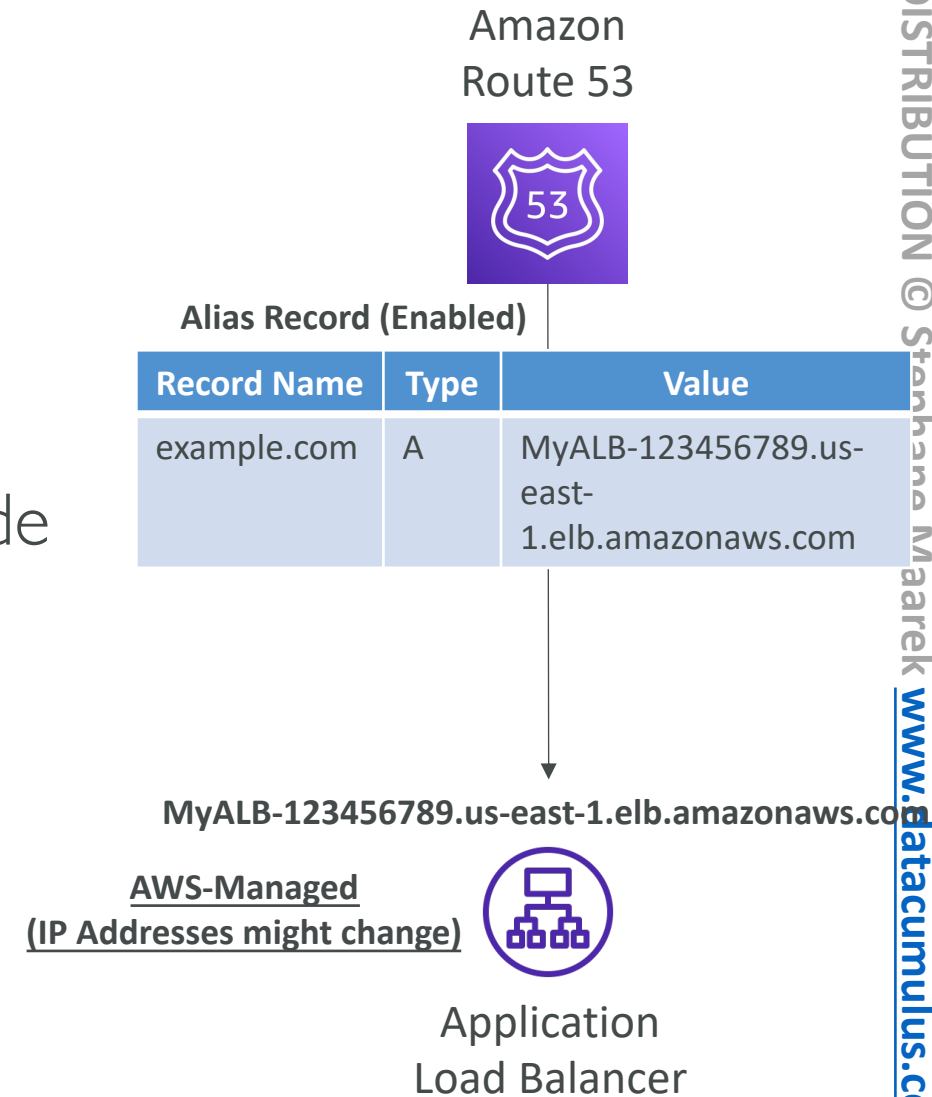


CNAME vs Alias

- AWS Resources (Load Balancer, CloudFront...) expose an AWS hostname:
 - [lb1-1234.us-east-2.elb.amazonaws.com](#) and you want [myapp.mydomain.com](#)
- CNAME:
 - Points a hostname to any other hostname. (app.mydomain.com => blabla.anything.com)
 - ONLY FOR NON ROOT DOMAIN (aka. something.mydomain.com)
- Alias:
 - Points a hostname to an AWS Resource (app.mydomain.com => blabla.amazonaws.com)
 - Works for ROOT DOMAIN and NON ROOT DOMAIN (aka mydomain.com)
 - Free of charge
 - Native health check

Route 53 – Alias Records

- Maps a hostname to an AWS resource
- An extension to DNS functionality
- Automatically recognizes changes in the resource's IP addresses
- Unlike CNAME, it can be used for the top node of a DNS namespace (Zone Apex), e.g.: example.com
- Alias Record is always of type A/AAAA for AWS resources (IPv4 / IPv6)
- You can't set the TTL



Route 53 – Alias Records Targets

- Elastic Load Balancers
- CloudFront Distributions
- API Gateway
- Elastic Beanstalk environments
- S3 Websites
- VPC Interface Endpoints
- Global Accelerator accelerator
- Route 53 record in the same hosted zone

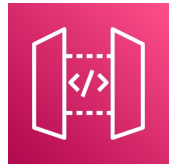
- You cannot set an ALIAS record for an EC2 DNS name



**Elastic
Load Balancer**



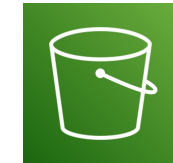
**Amazon
CloudFront**



**Amazon
API Gateway**



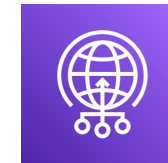
Elastic Beanstalk



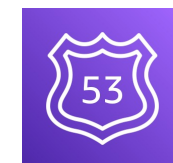
S3 Websites



**VPC Interface
Endpoints**



Global Accelerator



**Route 53 Record
(same Hosted Zone)**

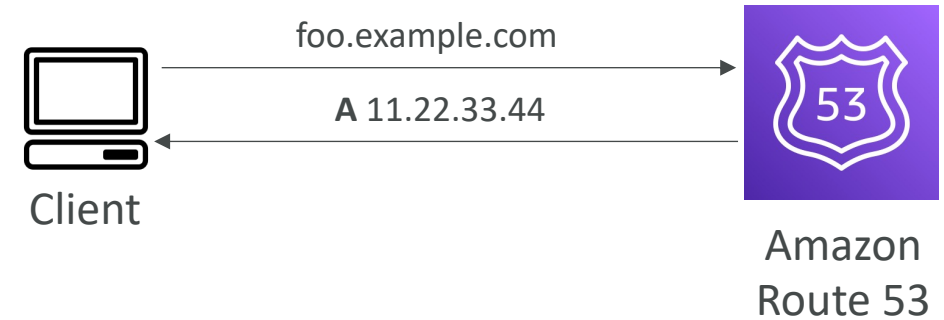
Route 53 – Routing Policies

- Define how Route 53 responds to DNS queries
- Don't get confused by the word "*Routing*"
 - It's not the same as Load balancer routing which routes the traffic
 - DNS does not route any traffic, it only responds to the DNS queries
- Route 53 Supports the following Routing Policies
 - Simple
 - Weighted
 - Failover
 - Latency based
 - Geolocation
 - Multi-Value Answer
 - Geoproximity (using Route 53 Traffic Flow feature)

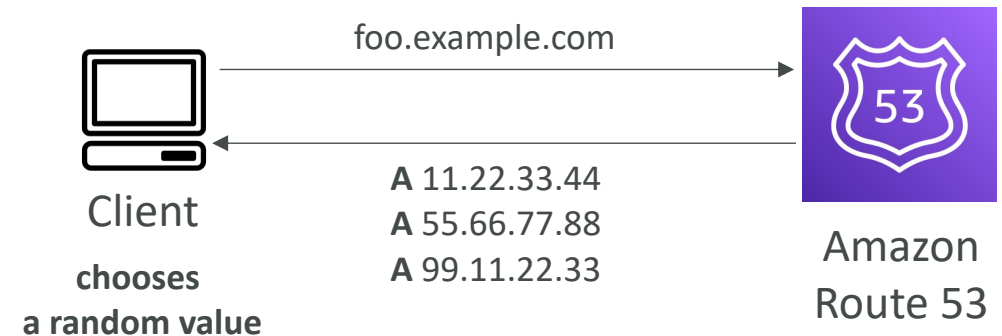
Routing Policies – Simple

- Typically, route traffic to a single resource
- Can specify multiple values in the same record
- If multiple values are returned, a random one is chosen by the client
- When Alias enabled, specify only one AWS resource
- Can't be associated with Health Checks

Single Value

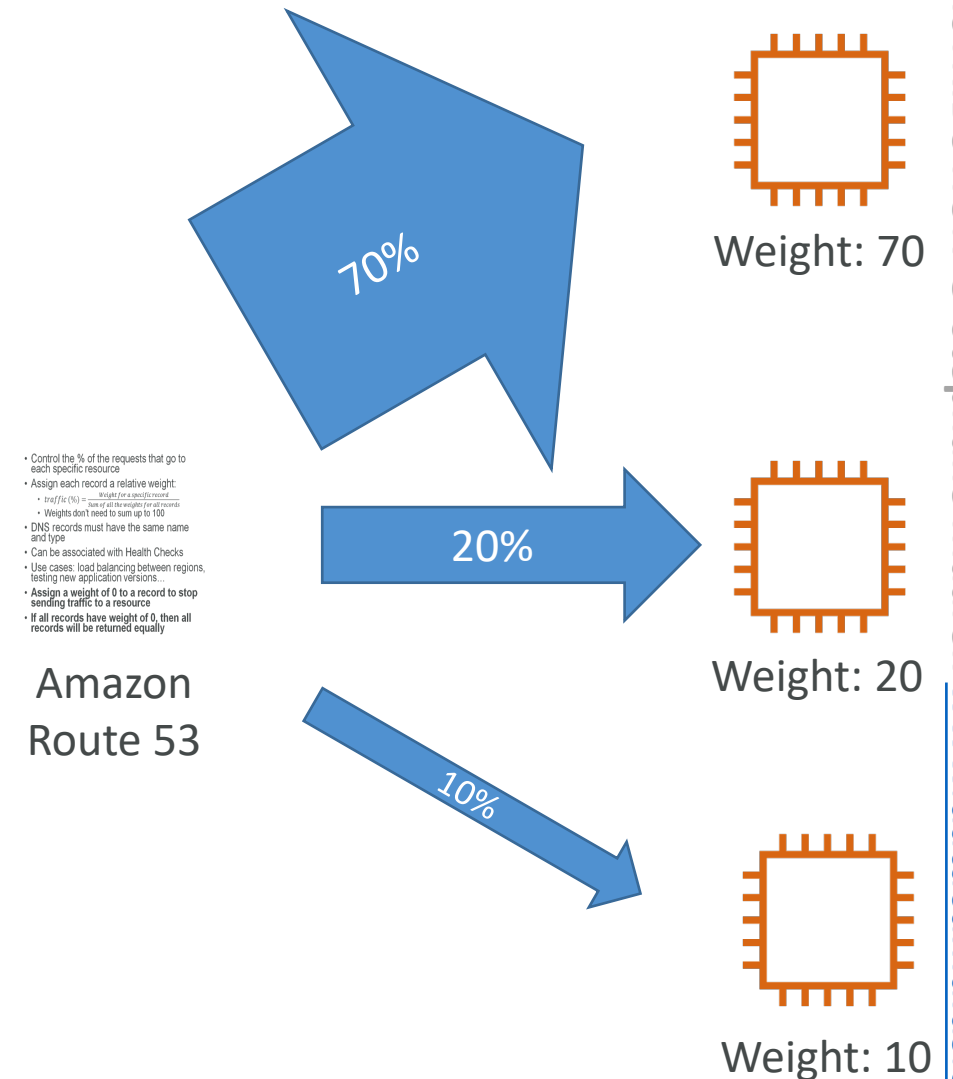


Multiple Value



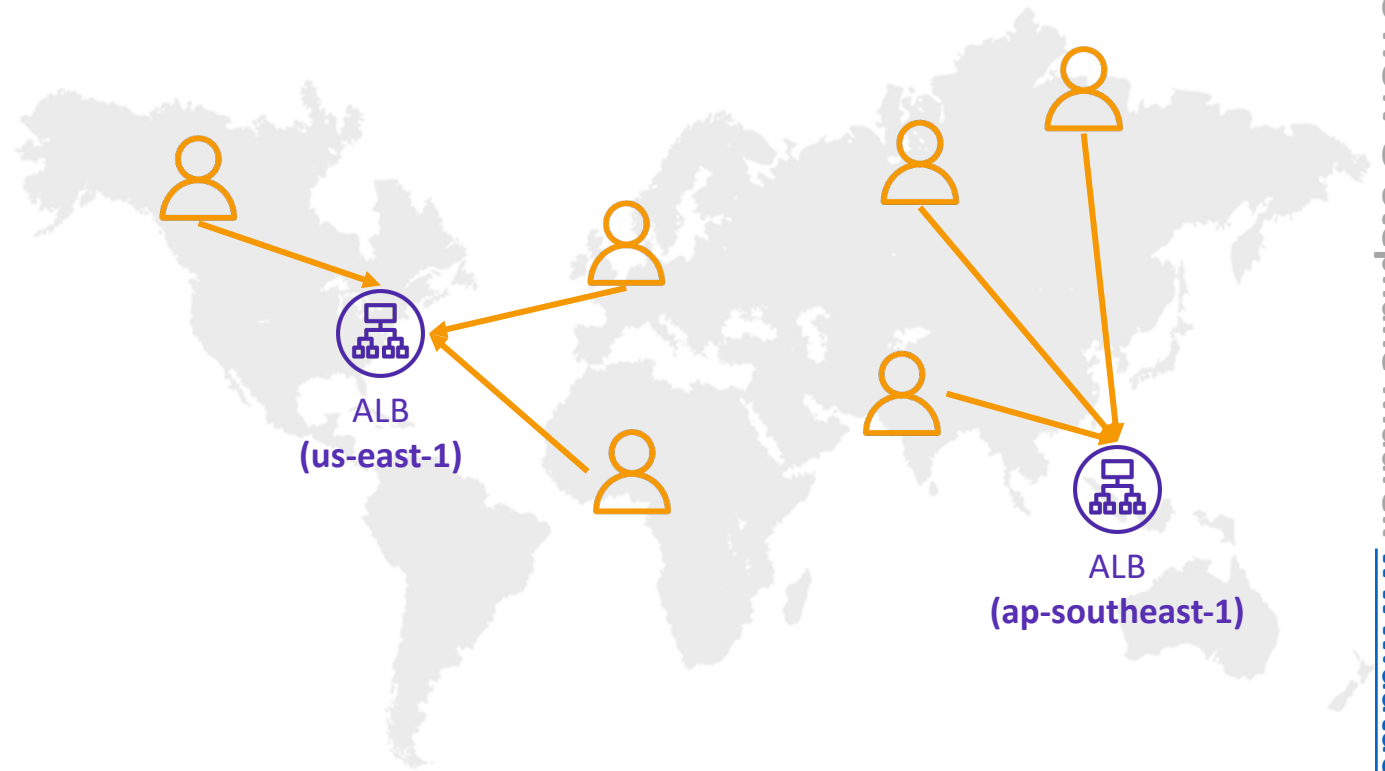
Routing Policies – Weighted

- Control the % of the requests that go to each specific resource
- Assign each record a relative weight:
 - $traffic\ (%) = \frac{Weight\ for\ a\ specific\ record}{Sum\ of\ all\ the\ weights\ for\ all\ records}$
 - Weights don't need to sum up to 100
- DNS records must have the same name and type
- Can be associated with Health Checks
- Use cases: load balancing between regions, testing new application versions...
- Assign a weight of 0 to a record to stop sending traffic to a resource
- If all records have weight of 0, then all records will be returned equally



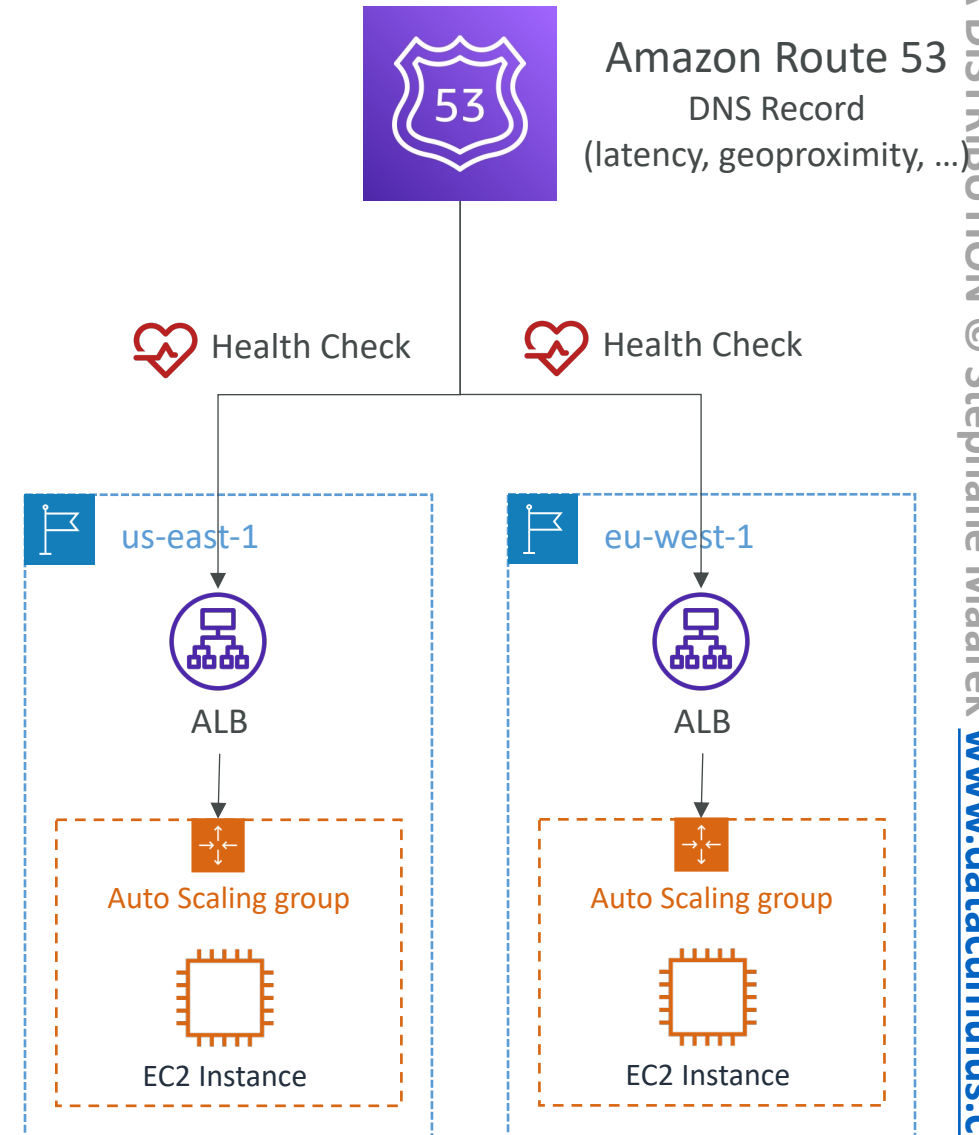
Routing Policies – Latency-based

- Redirect to the resource that has the least latency close to us
- Super helpful when latency for users is a priority
- Latency is based on traffic between users and AWS Regions
- Germany users may be directed to the US (if that's the lowest latency)
- Can be associated with Health Checks (has a failover capability)



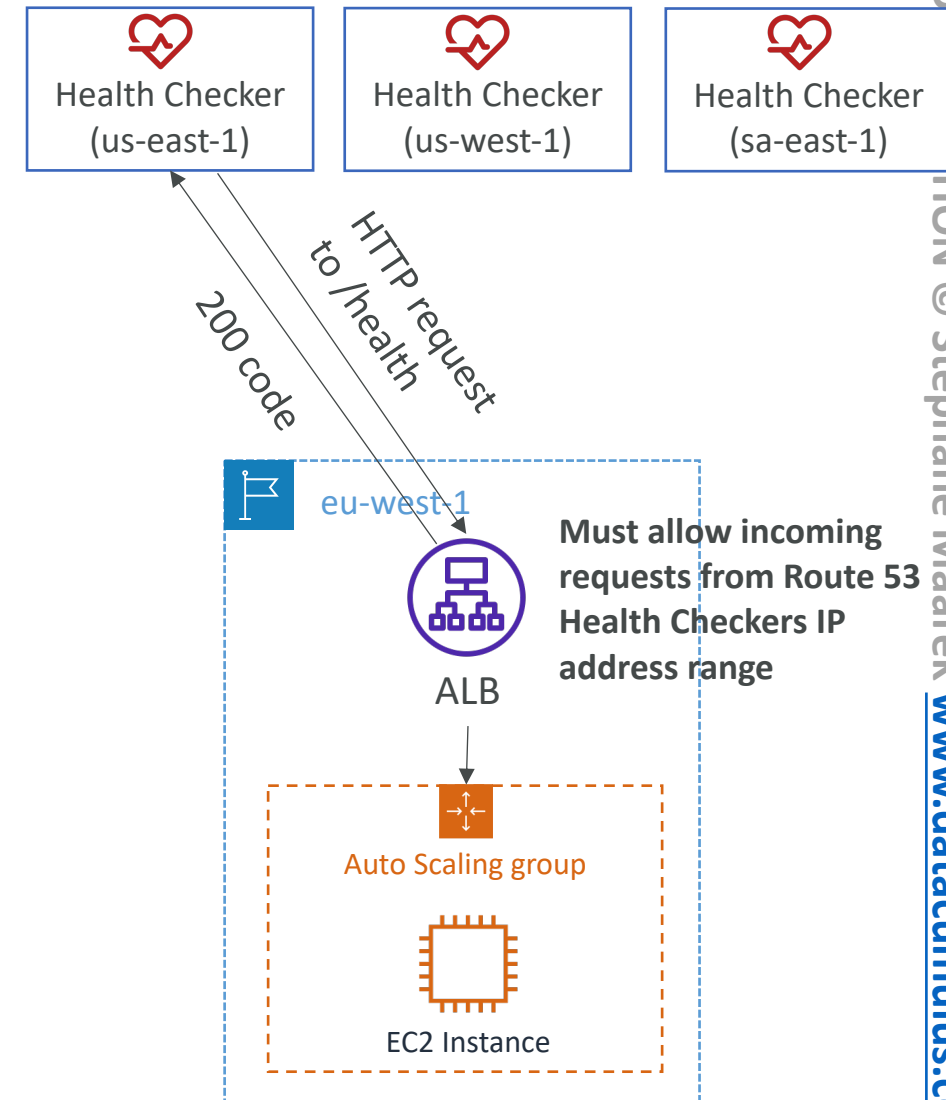
Route 53 – Health Checks

- HTTP Health Checks are only for **public resources**
- Health Check => Automated DNS Failover:
 1. Health checks that monitor an endpoint (application, server, other AWS resource)
 2. Health checks that monitor other health checks (Calculated Health Checks)
 3. Health checks that monitor CloudWatch Alarms (full control !!) – e.g., throttles of DynamoDB, alarms on RDS, custom metrics, ... (helpful for private resources)
- Health Checks are integrated with CW metrics



Health Checks – Monitor an Endpoint

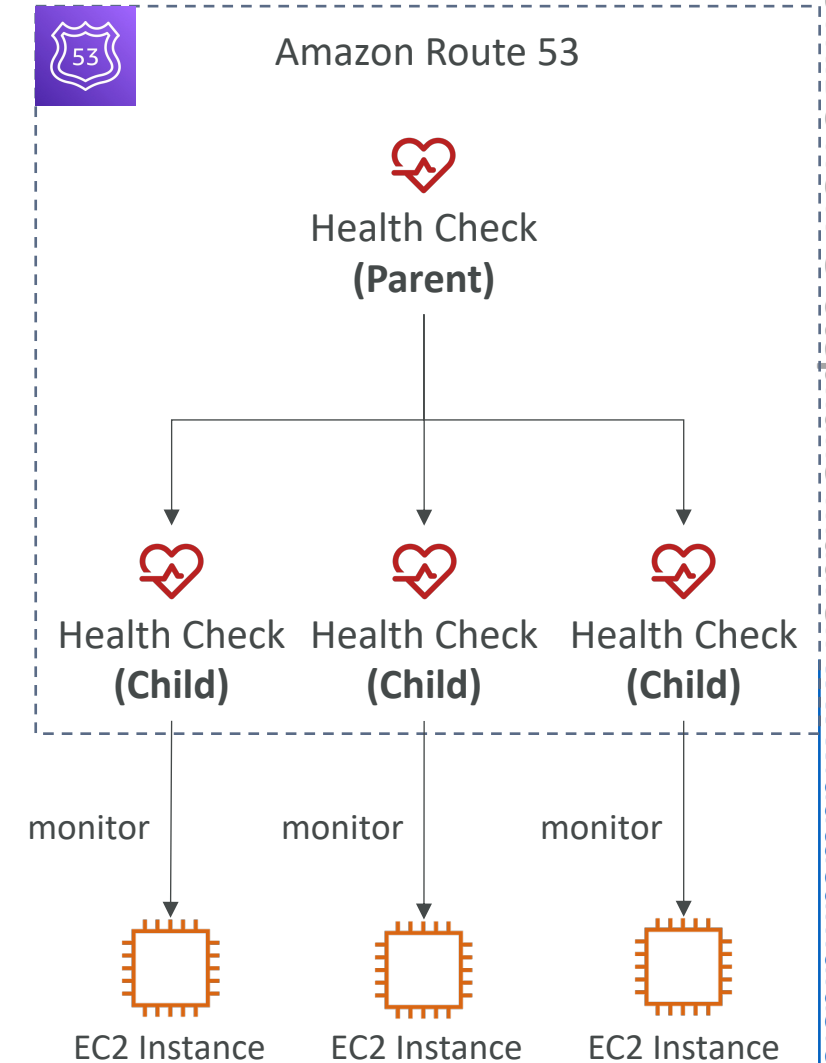
- About 15 global health checkers will check the endpoint health
 - Healthy/Unhealthy Threshold – 3 (default)
 - Interval – 30 sec (can set to 10 sec – higher cost)
 - Supported protocol: HTTP, HTTPS and TCP
 - If > 18% of health checkers report the endpoint is healthy, Route 53 considers it **Healthy**. Otherwise, it's **Unhealthy**
 - Ability to choose which locations you want Route 53 to use
- Health Checks pass only when the endpoint responds with the 2xx and 3xx status codes
- Health Checks can be setup to pass / fail based on the text in the first **5120 bytes** of the response
- Configure your router/firewall to allow incoming requests from Route 53 Health Checkers



<https://ip-ranges.amazonaws.com/ip-ranges.json>

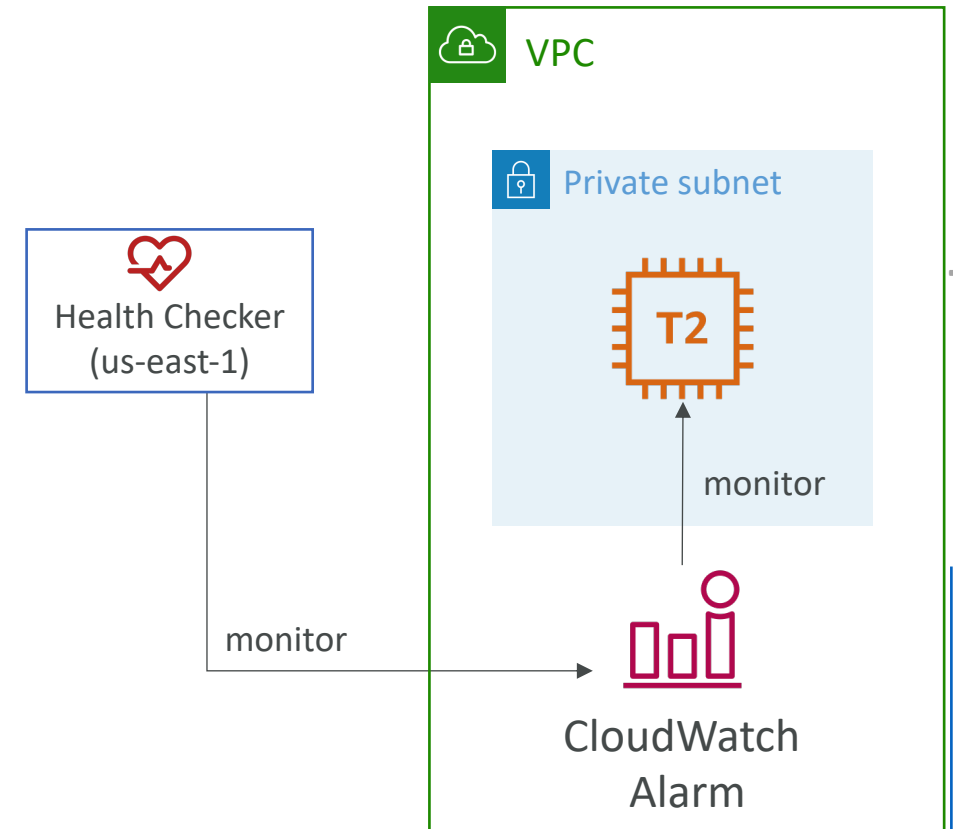
Route 53 – Calculated Health Checks

- Combine the results of multiple Health Checks into a single Health Check
- You can use **OR**, **AND**, or **NOT**
- Can monitor up to 256 Child Health Checks
- Specify how many of the health checks need to pass to make the parent pass
- Usage: perform maintenance to your website without causing all health checks to fail

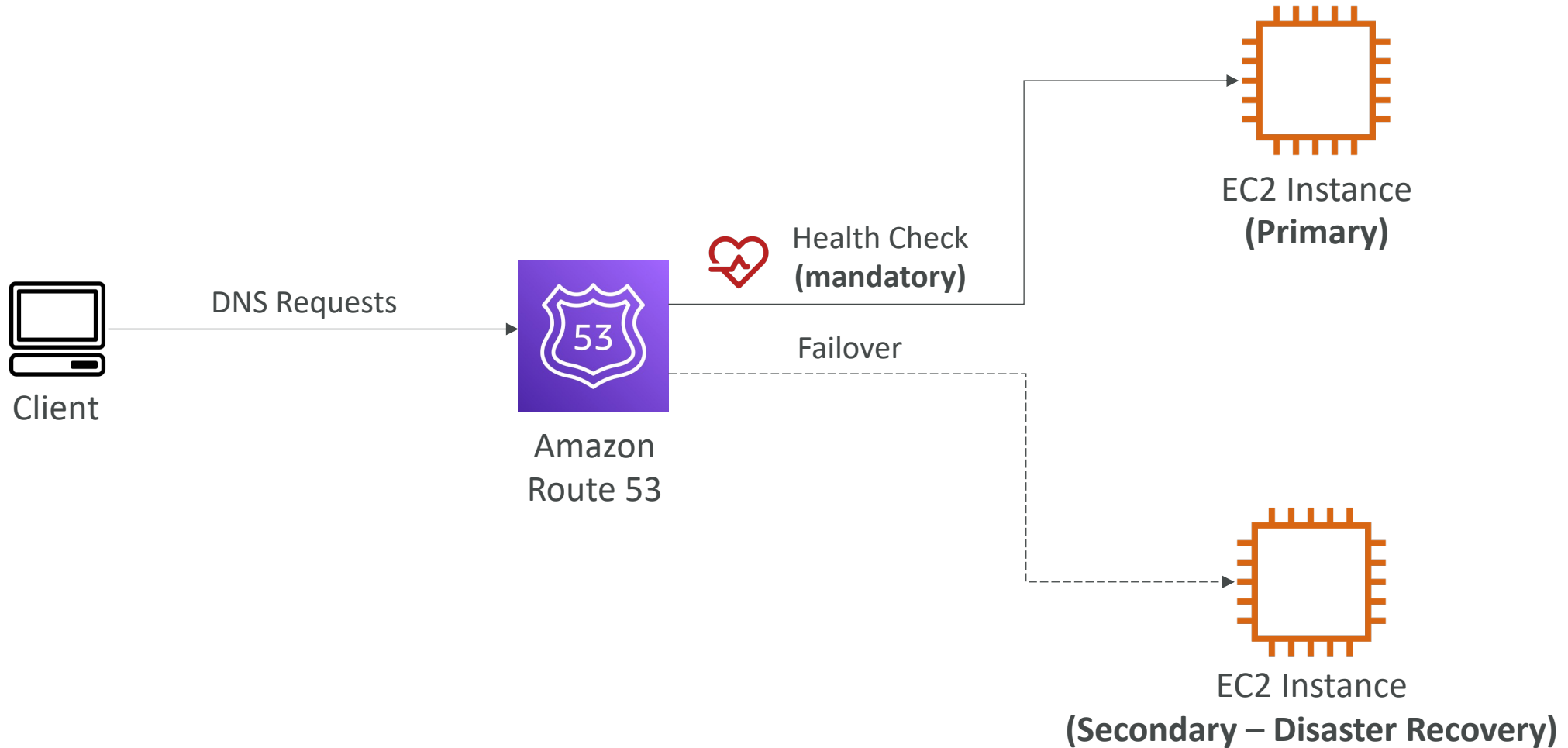


Health Checks – Private Hosted Zones

- Route 53 health checkers are outside the VPC
- They can't access **private** endpoints (private VPC or on-premises resource)
- You can create a **CloudWatch Metric** and associate a **CloudWatch Alarm**, then create a Health Check that checks the alarm itself

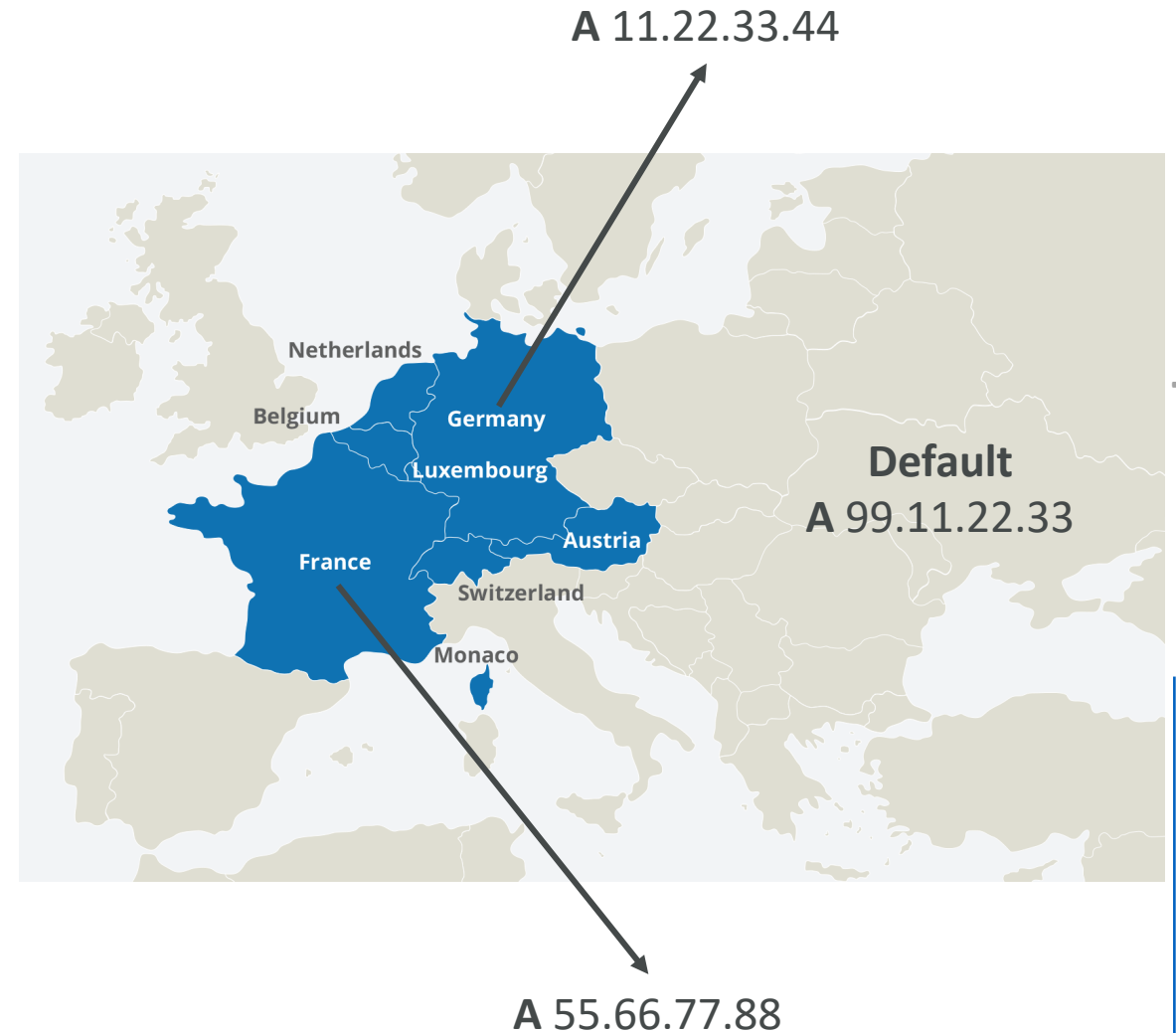


Routing Policies – Failover (Active-Passive)



Routing Policies – Geolocation

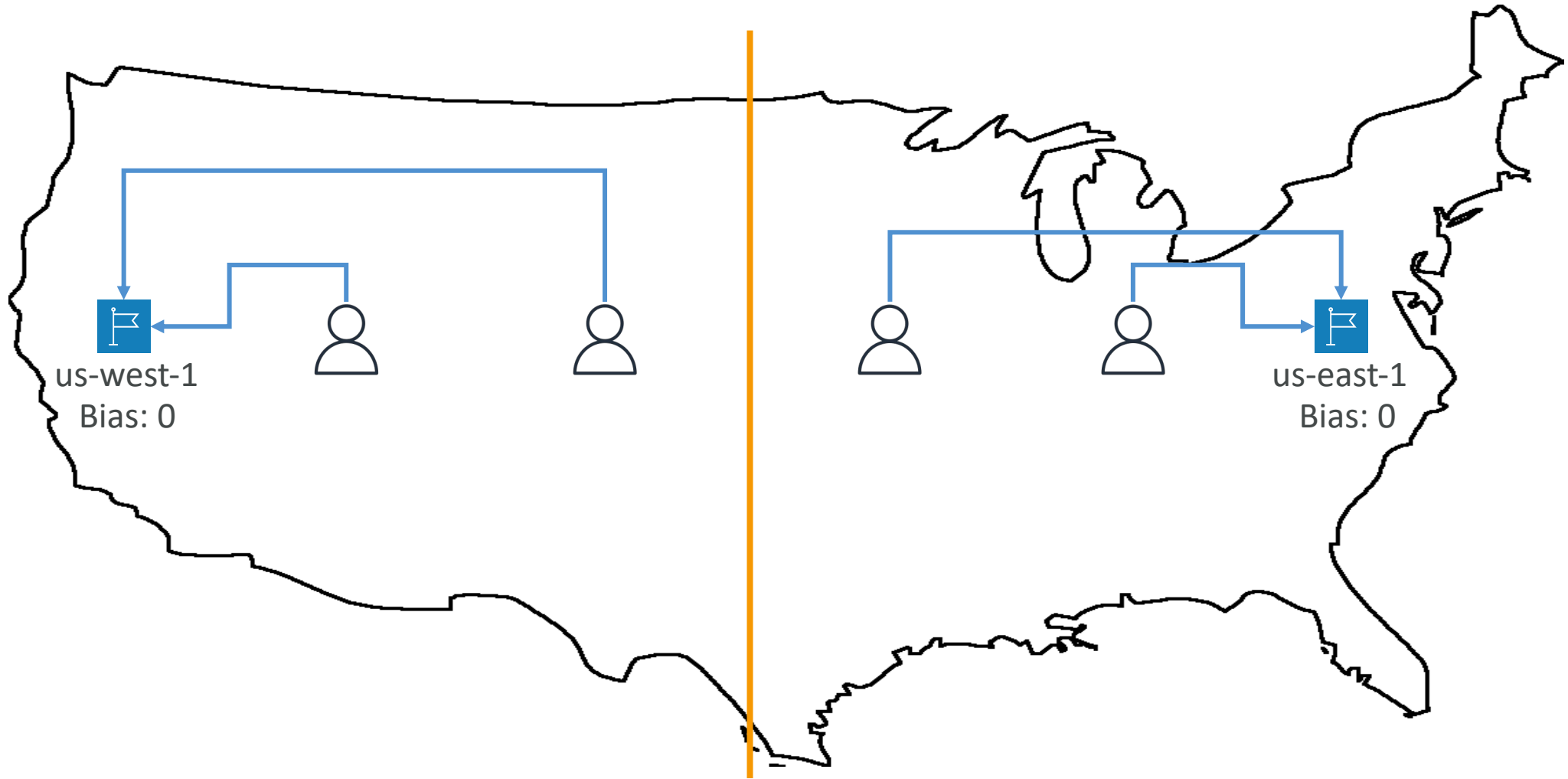
- Different from Latency-based!
- This routing is based on user location
- Specify location by Continent, Country or by US State (if there's overlapping, most precise location selected)
- Should create a “**Default**” record (in case there's no match on location)
- Use cases: website localization, restrict content distribution, load balancing, ...
- Can be associated with Health Checks



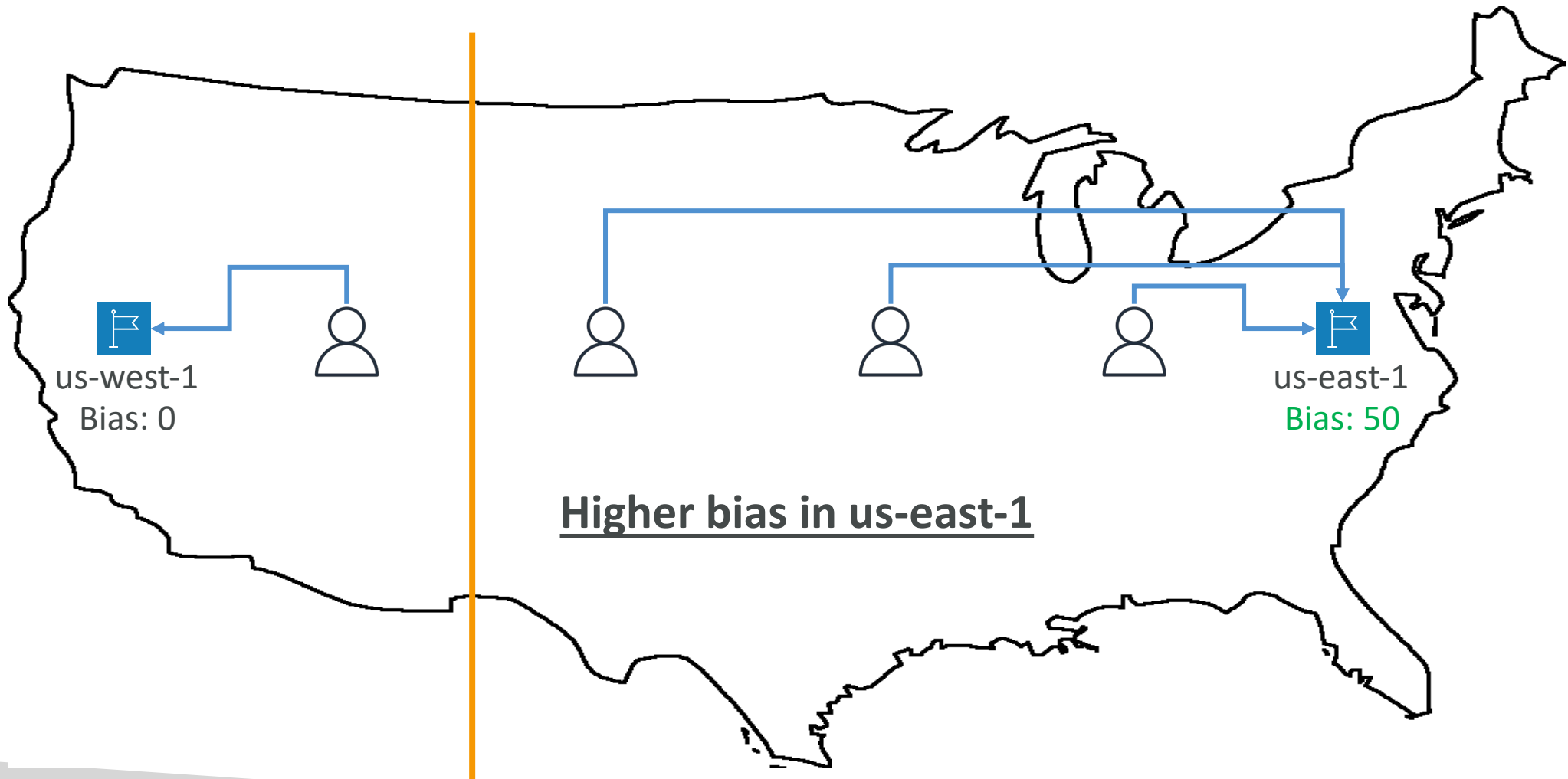
Routing Policies – Geoproximity

- Route traffic to your resources based on the geographic location of users and resources
- Ability **to shift more traffic to resources based** on the defined **bias**
- To change the size of the geographic region, specify **bias** values:
 - To expand (1 to 99) – more traffic to the resource
 - To shrink (-1 to -99) – less traffic to the resource
- Resources can be:
 - AWS resources (specify AWS region)
 - Non-AWS resources (specify Latitude and Longitude)
- You must use Route 53 **Traffic Flow** to use this feature

Routing Policies – Geoproximity

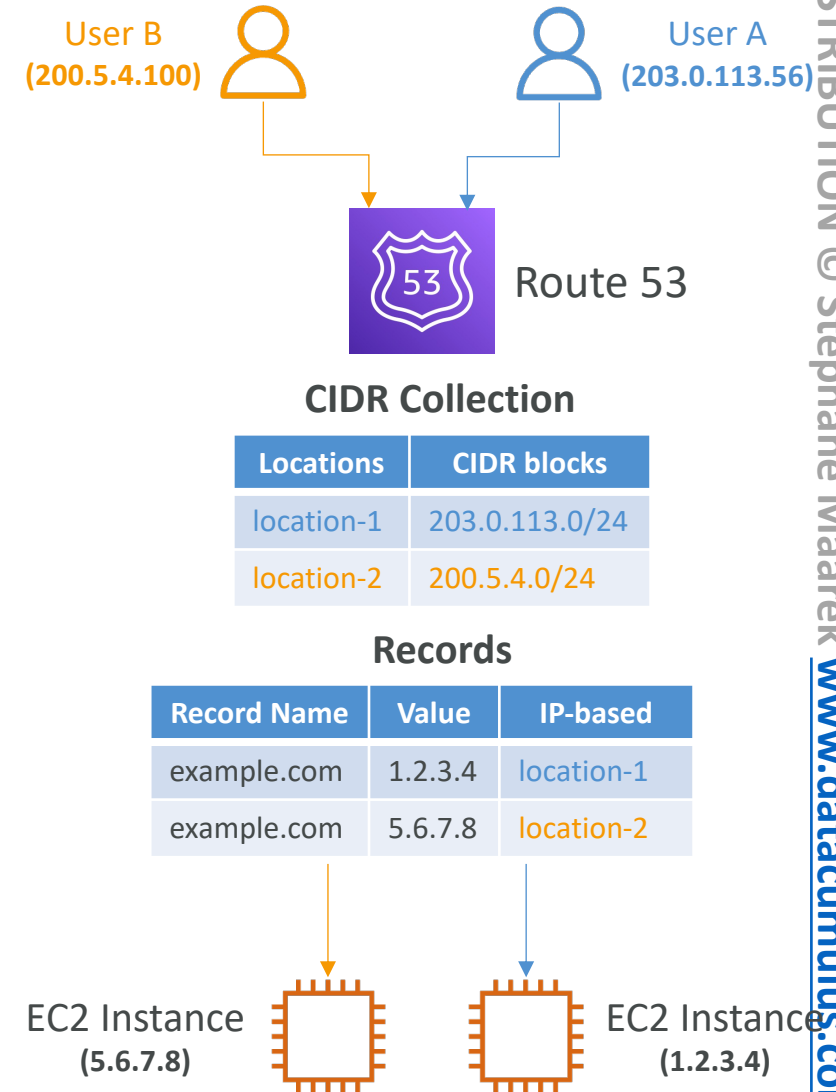


Routing Policies – Geoproximity



Routing Policies – IP-based Routing

- Routing is based on clients' IP addresses
- You provide a list of CIDRs for your clients and the corresponding endpoints/locations (user-IP-to-endpoint mappings)
- Use cases: Optimize performance, reduce network costs...
- Example: route end users from a particular ISP to a specific endpoint



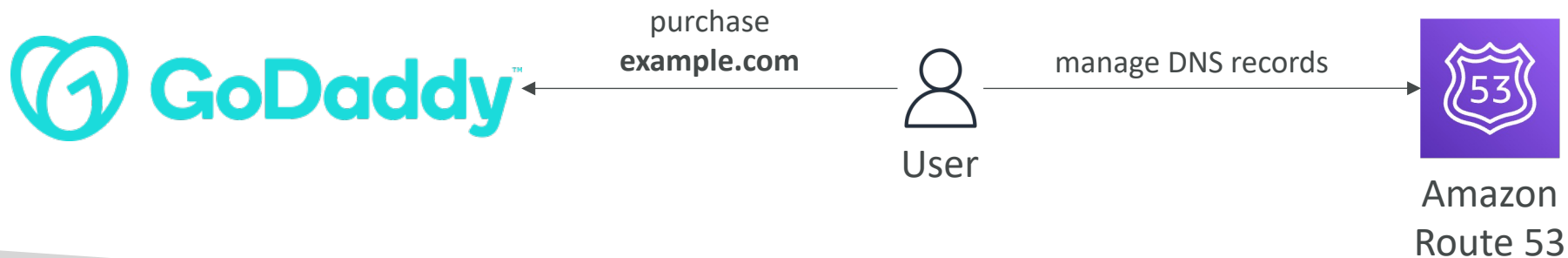
Routing Policies – Multi-Value

- Use when routing traffic to multiple resources
- Route 53 return multiple values/resources
- Can be associated with Health Checks (return only values for healthy resources)
- Up to 8 healthy records are returned for each Multi-Value query
- Multi-Value is not a substitute for having an ELB

Name	Type	Value	TTL	Set ID	Health Check
www.example.com	A Record	192.0.2.2	60	Web1	A
www.example.com	A Record	198.51.100.2	60	Web2	B
www.example.com	A Record	203.0.113.2	60	Web3	C

Domain Registrar vs. DNS Service

- You buy or register your domain name with a Domain Registrar typically by paying annual charges (e.g., GoDaddy, Amazon Registrar Inc., ...)
- The Domain Registrar usually provides you with a DNS service to manage your DNS records
- But you can use another DNS service to manage your DNS records
- Example: purchase the domain from GoDaddy and use Route 53 to manage your DNS records



GoDaddy as Registrar & Route 53 as DNS Service



Records

We can't display your DNS information because your nameservers aren't managed by us.

Nameservers

Using custom nameservers [Change](#)

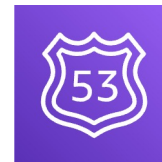
Nameserver

ns-1083.awsdns-07.org

ns-932.awsdns-52.net

ns-1911.awsdns-46.co.uk

ns-481.awsdns-60.com



Amazon
Route 53

Public Hosted Zone
stephanetheteacher.com

Hosted zone details [Edit hosted zone](#)

Hosted zone ID Z30IUJCCWPKZUV	Type Public hosted zone
Description HostedZone created by Route53 Registrar	Record count 22
Query log	

Name servers
ns-252.awsdns-31.com
ns-1468.awsdns-55.org
ns-633.awsdns-15.net
ns-1800.awsdns-33.co.uk

3rd Party Registrar with Amazon Route 53

- If you buy your domain on a 3rd party registrar, you can still use Route 53 as the DNS Service provider
 1. Create a Hosted Zone in Route 53
 2. Update NS Records on 3rd party website to use Route 53 **Name Servers**
- Domain Registrar != DNS Service
- But every Domain Registrar usually comes with some DNS features

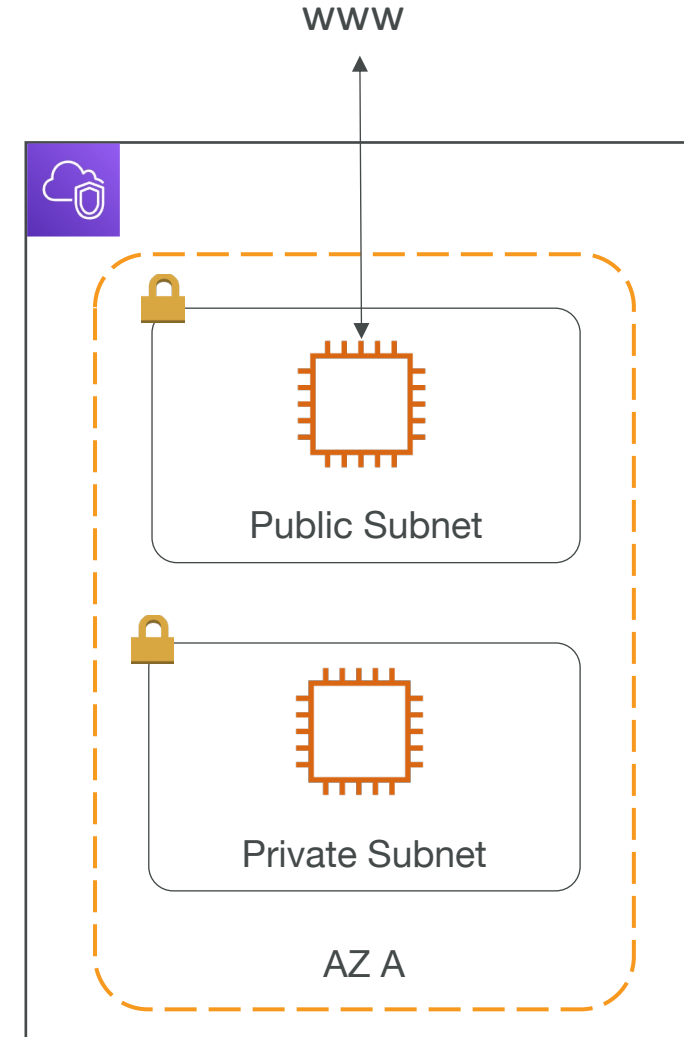
Amazon VPC – Basics

VPC – Crash Course

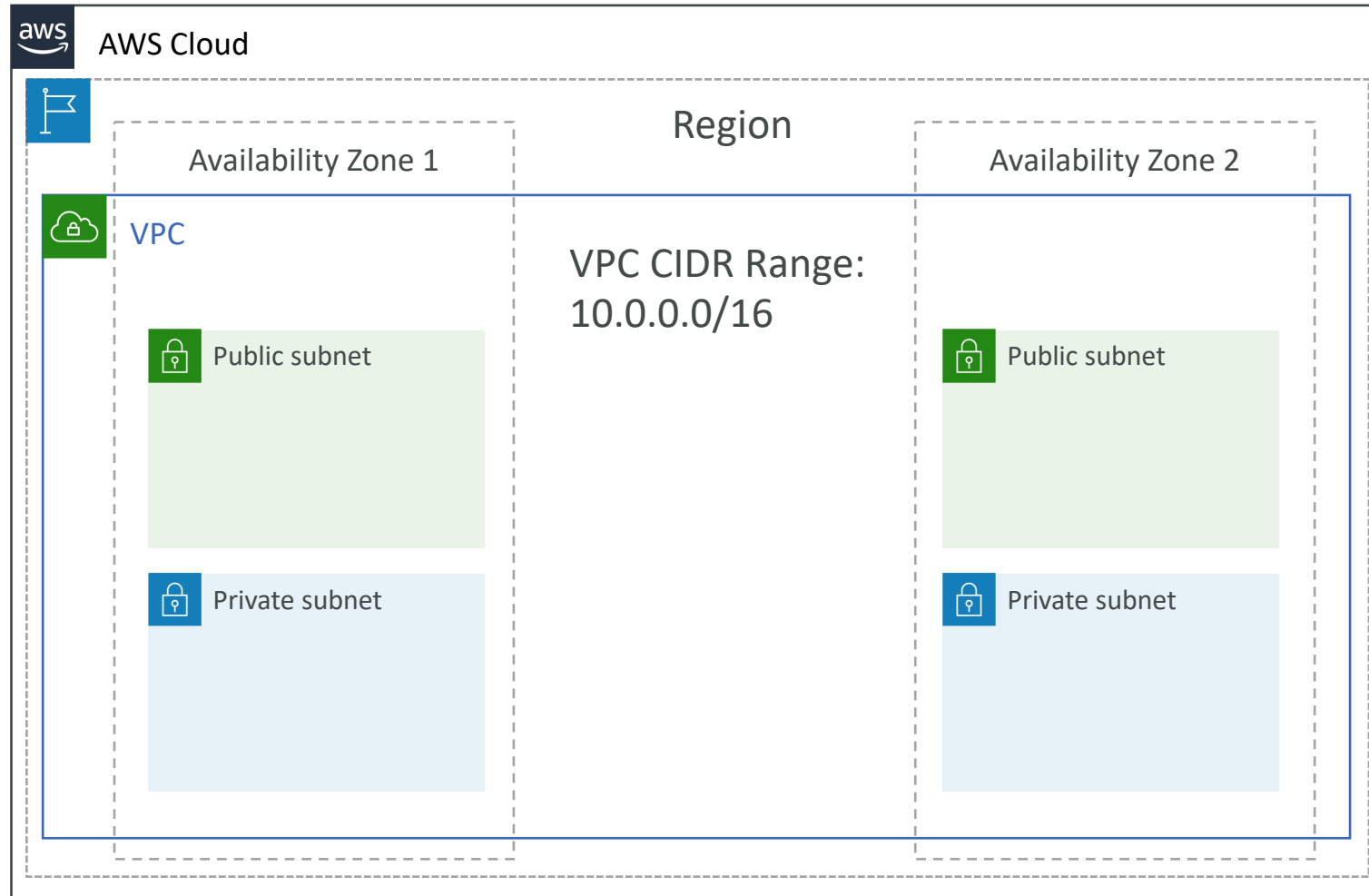
- VPC is something you should know in depth for the AWS Certified Solutions Architect Associate & AWS Certified SysOps Administrator
- **At the AWS Certified Developer Level, you should know about:**
 - VPC, Subnets, Internet Gateways & NAT Gateways
 - Security Groups, Network ACL (NACL), VPC Flow Logs
 - VPC Peering, VPC Endpoints
 - Site to Site VPN & Direct Connect
- I will just give you an overview, less than 1 or 2 questions at your exam.
- Later in the course, I will be highlighting when VPC concepts are helpful

VPC & Subnets Primer

- **VPC**: private network to deploy your resources (regional resource)
- **Subnets** allow you to partition your network inside your VPC (Availability Zone resource)
- A **public subnet** is a subnet that is accessible from the internet
- A **private subnet** is a subnet that is not accessible from the internet
- To define access to the internet and between subnets, we use **Route Tables**.

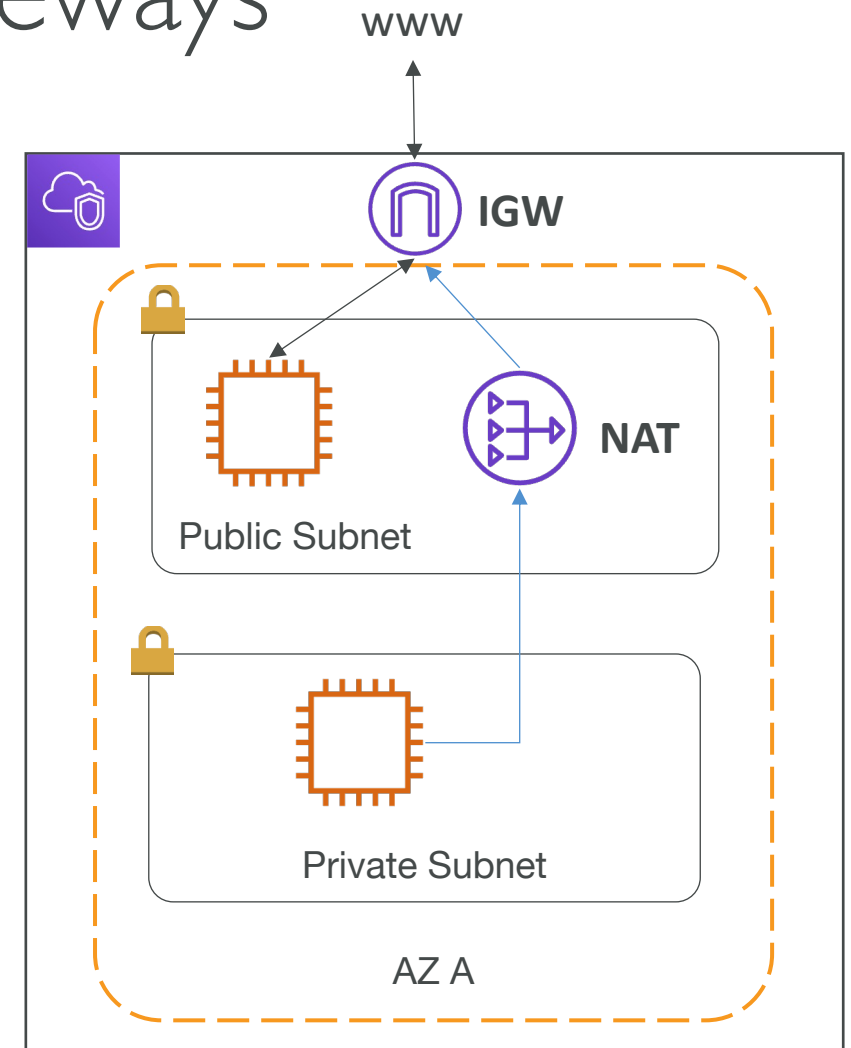


VPC Diagram



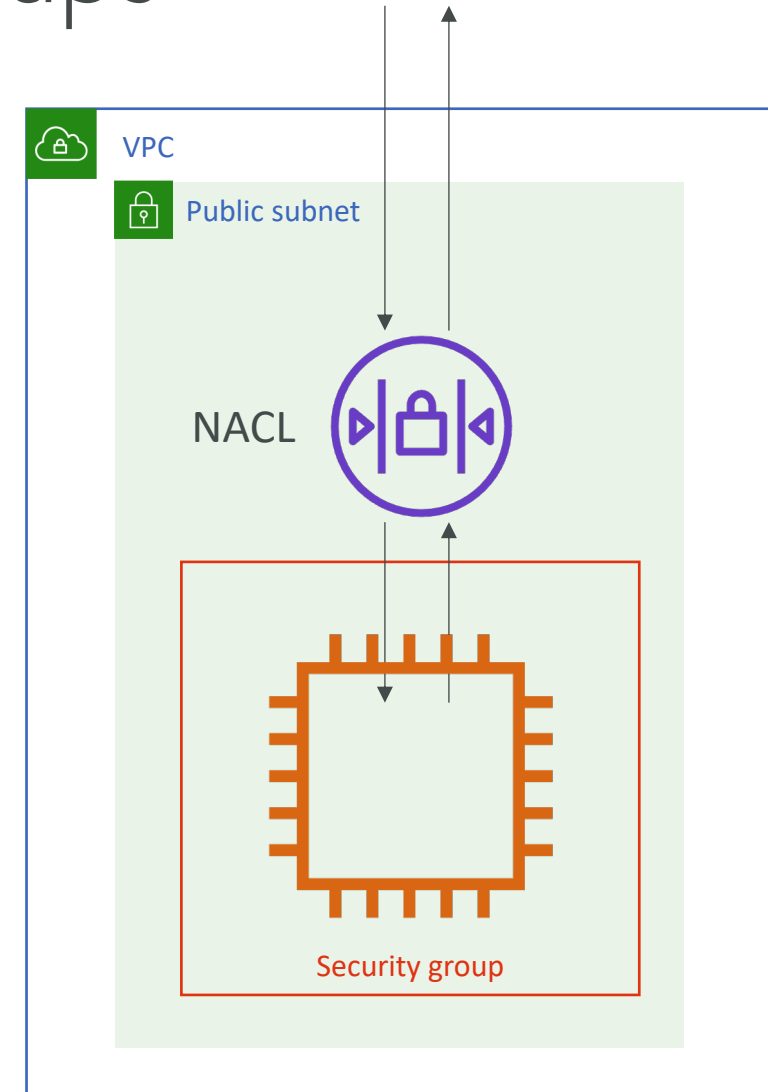
Internet Gateway & NAT Gateways

- **Internet Gateways** helps our VPC instances connect with the internet
- Public Subnets have a route to the internet gateway.
- **NAT Gateways** (AWS-managed) & **NAT Instances** (self-managed) allow your instances in your **Private Subnets** to access the internet while remaining private



Network ACL & Security Groups

- **NACL** (Network ACL)
 - A firewall which controls traffic from and to subnet
 - Can have ALLOW and DENY rules
 - Are attached at the **Subnet** level
 - Rules only include IP addresses
- **Security Groups**
 - A firewall that controls traffic to and from **an ENI / an EC2 Instance**
 - Can have only ALLOW rules
 - Rules include IP addresses and other security groups



Network ACLs vs Security Groups

Security Group	Network ACL
Operates at the instance level	Operates at the subnet level
Supports allow rules only	Supports allow rules and deny rules
Is stateful: Return traffic is automatically allowed, regardless of any rules	Is stateless: Return traffic must be explicitly allowed by rules
We evaluate all rules before deciding whether to allow traffic	We process rules in number order when deciding whether to allow traffic
Applies to an instance only if someone specifies the security group when launching the instance, or associates the security group with the instance later on	Automatically applies to all instances in the subnets it's associated with (therefore, you don't have to rely on users to specify the security group)

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Security.html#VPC_Security_Comparison

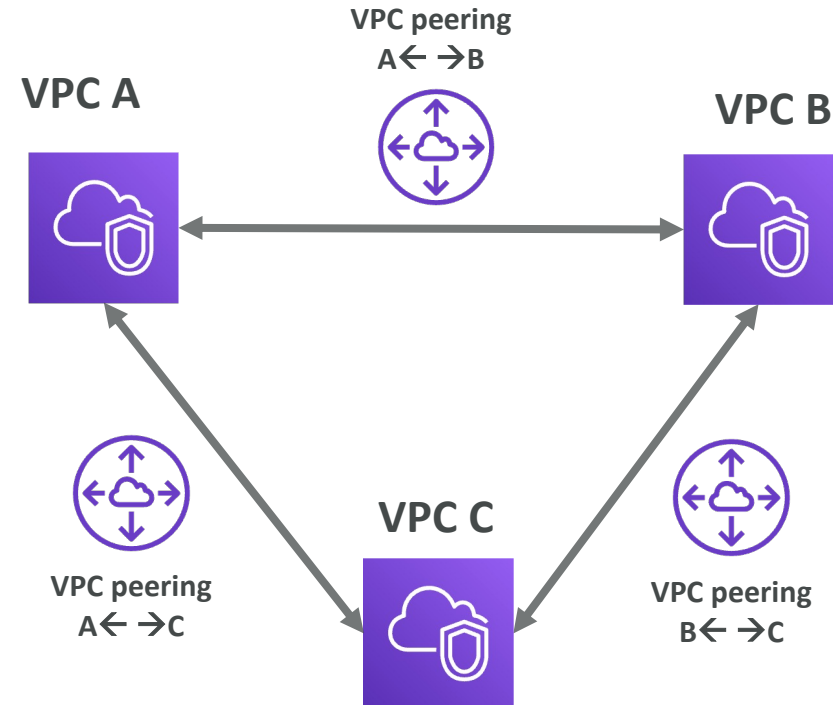


VPC Flow Logs

- Capture information about IP traffic going into your interfaces:
 - VPC Flow Logs
 - Subnet Flow Logs
 - Elastic Network Interface Flow Logs
- Helps to monitor & troubleshoot connectivity issues. Example:
 - Subnets to internet
 - Subnets to subnets
 - Internet to subnets
- Captures network information from AWS managed interfaces too: Elastic Load Balancers, ElastiCache, RDS, Aurora, etc...
- VPC Flow logs data can go to S3, CloudWatch Logs, and Kinesis Data Firehose

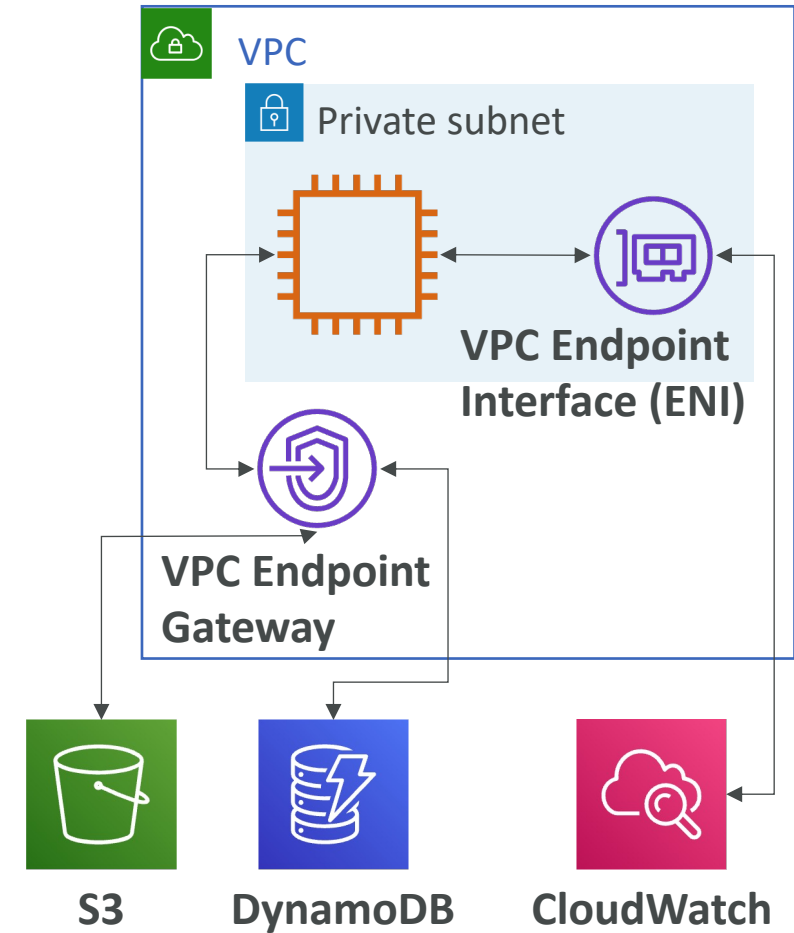
VPC Peering

- Connect two VPC, privately using AWS' network
- Make them behave as if they were in the same network
- Must not have overlapping CIDR (IP address range)
- VPC Peering connection is **not transitive** (must be established for each VPC that need to communicate with one another)



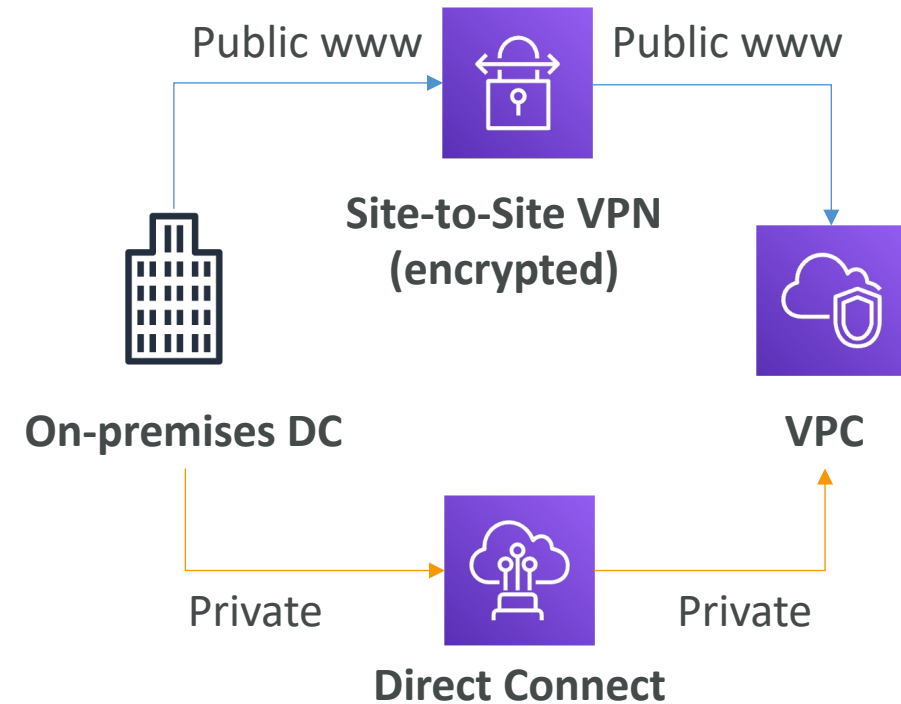
VPC Endpoints

- Endpoints allow you to connect to AWS Services **using a private network** instead of the public www network
- This gives you enhanced security and lower latency to access AWS services
- VPC Endpoint Gateway: S3 & DynamoDB
- VPC Endpoint Interface: most services (including S3 & DynamoDB)
- Only used within your VPC



Site to Site VPN & Direct Connect

- Site to Site VPN
 - Connect an on-premises VPN to AWS
 - The connection is automatically encrypted
 - Goes over the public internet
- Direct Connect (DX)
 - Establish a physical connection between on-premises and AWS
 - The connection is private, secure and fast
 - Goes over a private network
 - Takes at least a month to establish



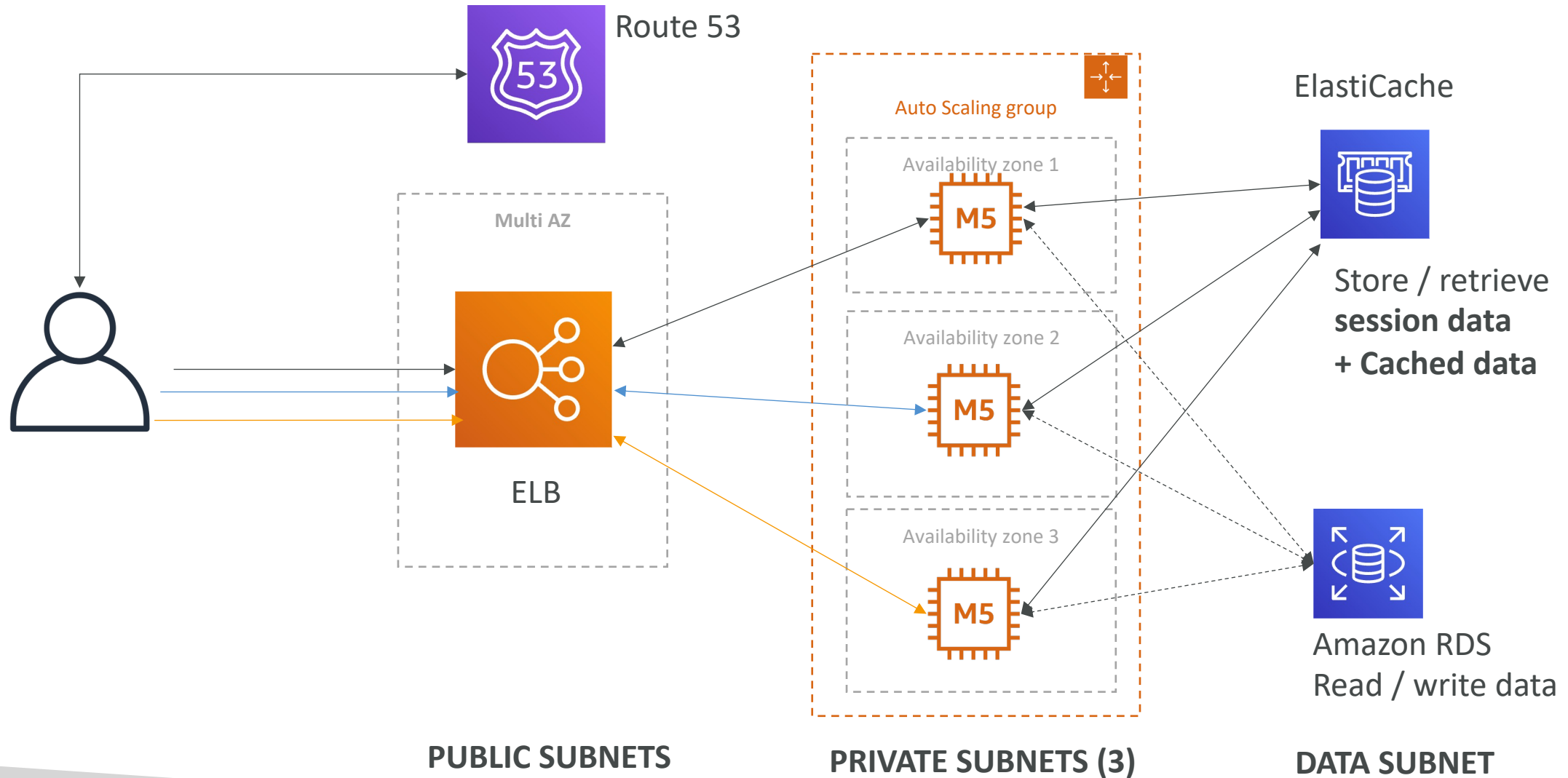
VPC Closing Comments

- **VPC:** Virtual Private Cloud
- **Subnets:** Tied to an AZ, network partition of the VPC
- **Internet Gateway:** at the VPC level, provide Internet Access
- **NAT Gateway / Instances:** give internet access to private subnets
- **NACL:** Stateless, subnet rules for inbound and outbound
- **Security Groups:** Stateful, operate at the EC2 instance level or ENI
- **VPC Peering:** Connect two VPC with non overlapping IP ranges, non transitive
- **VPC Endpoints:** Provide private access to AWS Services within VPC
- **VPC Flow Logs:** network traffic logs
- **Site to Site VPN:** VPN over public internet between on-premises DC and AWS
- **Direct Connect:** direct private connection to a AWS

VPC note – AWS Certified Developer

- Don't stress if you didn't understand everything in that section
- I will be highlighting in the course the specific VPC features we need
- Feel free to revisit that section after you're done in the course !
- Moving on 😊

Typical 3 tier solution architecture

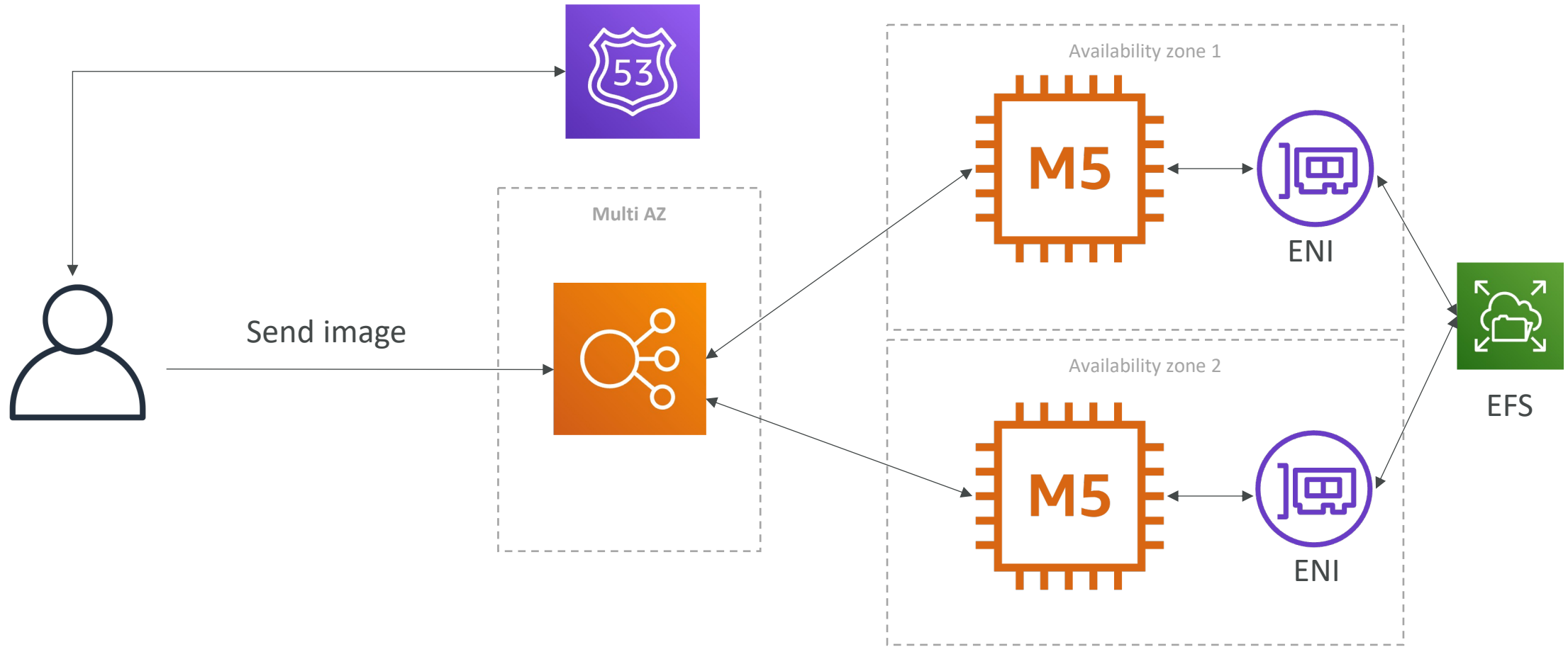


LAMP Stack on EC2

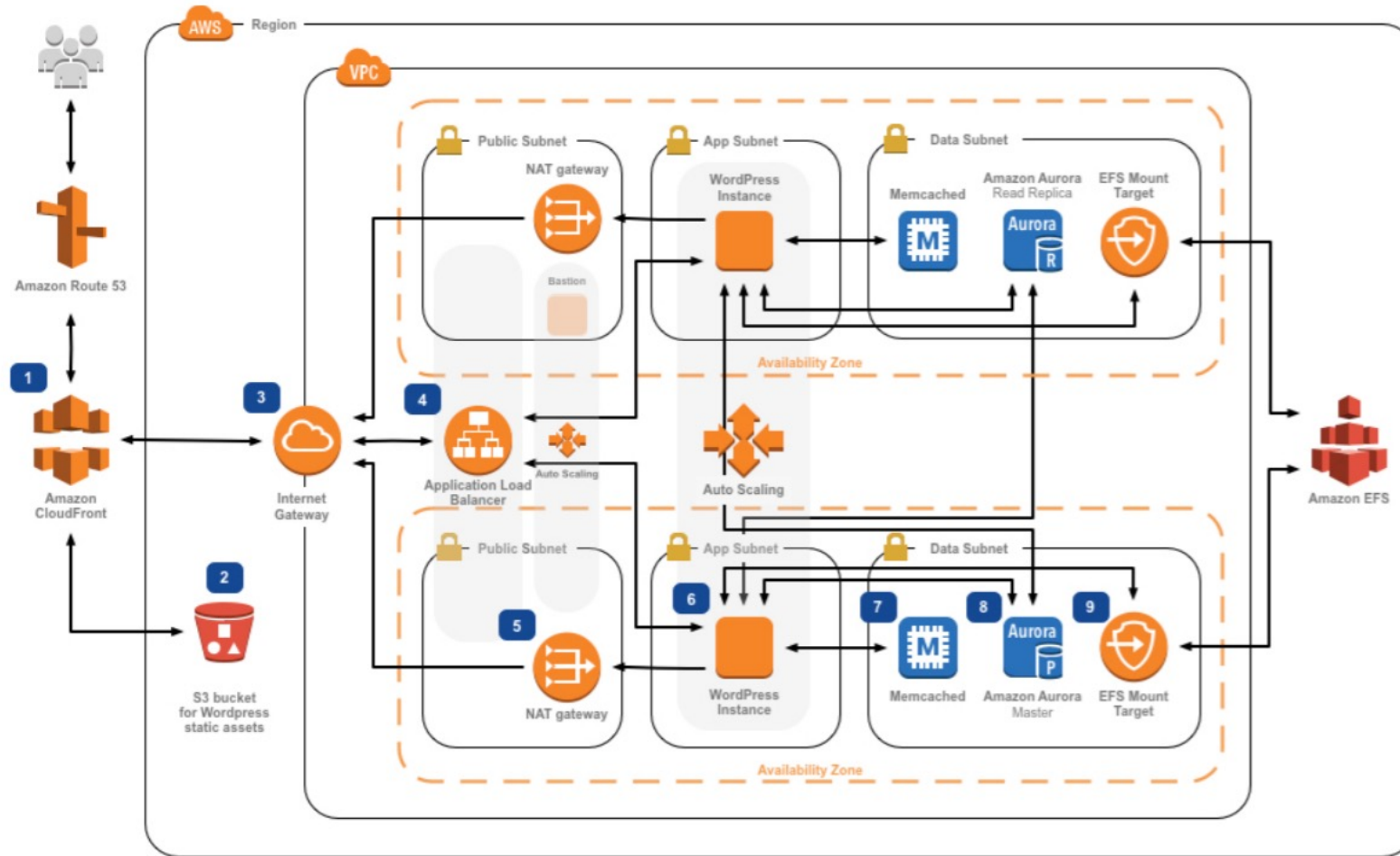
- Linux: OS for EC2 instances
- Apache: Web Server that run on Linux (EC2)
- MySQL: database on RDS
- PHP: Application logic (running on EC2)

- Can add Redis / Memcached (**ElastiCache**) to include a caching tech
- To store local application data & software: **EBS** drive (root)

Wordpress on AWS



WordPress on AWS (more complicated)



<https://aws.amazon.com/blogs/architecture/wordpress-best-practices-on-aws/>